

Unit III

Functions, Lists & Basic Algorithms

Python Programming | Academic Notes

Topics Covered

Functions | Arguments & Return Values | Lists | List Methods | Algorithms

1. Introduction to Functions

What is a Function?

A **function** is a named block of code that performs a specific task. Instead of writing the same code again and again, you write it once inside a function and simply *call* it whenever you need it.

Why Use Functions? (Advantages)

Advantage	What it means
Reusability	Write once, use many times anywhere in the program.
Readability	Code is easier to read and understand when split into small functions.
Modularity	Break a big problem into smaller, manageable pieces.
Easy Debugging	Fix a bug in one place – it is fixed everywhere the function is used.
Avoid Repetition	No need to copy-paste the same code multiple times.

■ **Quick Tip:** Think of a function like a blender. You put ingredients in (input), it does the work, and gives you a smoothie out (output). You don't need to know how it works inside every time!

2. Defining and Calling Functions

Syntax to Define a Function

```
def function_name(parameters):    # def keyword + name + parentheses

    '''Optional description'''    # docstring (good practice!)

    # code / statements
```

return value

optional – sends value back

Example 1 – Simple Function with No Parameters

```
def greet():                                # defining the function

    print('Hello! Welcome.')                # body

greet()                                    # calling the function

# Output: Hello! Welcome.
```

Example 2 – Function with Parameters

```
def greet_user(name):                       # 'name' is a parameter

    print('Hello, ', name)

greet_user('Alice')                         # 'Alice' is the argument

greet_user('Bob')

# Output:

# Hello, Alice

# Hello, Bob
```

■ **Remember: Parameter vs Argument:** A *parameter* is the variable inside the function definition. An *argument* is the actual value you pass when calling it.

3. Function Arguments and Return Values

Types of Arguments

Type	Description	Example
Positional	Values matched by order.	add(3, 5)
Keyword	Values matched by name.	add(a=3, b=5)
Default	Parameter has a preset value if not provided.	def add(a, b=10)
Multiple (*args)	Accepts any number of positional args.	def total(*nums)

The return Statement

A function can **send a result back** to the caller using the `return` keyword. Once `return` runs, the function stops.

```
def add(a, b):
    result = a + b
    return result          # sends value back

total = add(10, 20)       # total = 30
print(total)              # Output: 30
```

Returning Multiple Values

```
def min_max(numbers):
    return min(numbers), max(numbers)  # returns a tuple

low, high = min_max([3, 1, 9, 2])
print(low, high)  # Output: 1 9
```

■ **Remember:** If a function has no `return` statement, it automatically returns `None`.

4. Introduction to Lists

What is a List?

A **list** is a collection that stores **multiple values in a single variable**. Lists are *ordered* (items have a fixed position), *mutable* (can be changed), and can hold items of *any data type*.

```
fruits = ['apple', 'banana', 'mango']  # list of strings
nums   = [10, 20, 30, 40]              # list of numbers
mixed  = [1, 'hello', 3.14, True]      # mixed types allowed
```

List Indexing

Each item in a list has an **index** starting from 0. You can also use **negative indexing** to access items from the end.

List	fruits = ['apple', 'banana', 'mango', 'grape']			
Index (forward)	0	1	2	3
Index (backward)	-4	-3	-2	-1

```
print(fruits[0])    # apple    (first item)

print(fruits[-1])   # grape    (last item)

print(fruits[1:3])  # ['banana', 'mango'] (slicing)
```

Basic List Operations

Operation	Syntax	Result / Effect
Length	<code>len(fruits)</code>	Number of items
Concatenate	<code>[1,2] + [3,4]</code>	<code>[1, 2, 3, 4]</code>
Repeat	<code>[0] * 3</code>	<code>[0, 0, 0]</code>
Check membership	<code>'apple' in fruits</code>	True or False
Iterate	<code>for x in fruits:</code>	Loops over each item

Operation	Syntax	Result / Effect
Length	<code>len(fruits)</code>	Number of items in the list
Concatenate	<code>[1, 2] + [3, 4]</code>	<code>[1, 2, 3, 4]</code>
Repeat	<code>[0] * 3</code>	<code>[0, 0, 0]</code>
Check member	<code>'apple' in fruits</code>	Returns True or False
Loop	<code>for x in fruits: ...</code>	Visits each item one by one

5. Basic List Methods

Python provides built-in **methods** (actions) that you can perform directly on a list. They are called using the dot (.) notation: `list_name.method()`.

Method	What it does	Example
<code>append(x)</code>	Adds x to the END of the list.	<code>fruits.append('kiwi')</code>
<code>insert(i, x)</code>	Adds x at position i.	<code>fruits.insert(1, 'peach')</code>
<code>remove(x)</code>	Removes the first occurrence of x.	<code>fruits.remove('banana')</code>
<code>pop(i)</code>	Removes and returns item at index i (last if no index).	<code>fruits.pop(0)</code>
<code>sort()</code>	Sorts the list in ascending order.	<code>nums.sort()</code>
<code>reverse()</code>	Reverses the list in place.	<code>nums.reverse()</code>
<code>index(x)</code>	Returns index of first occurrence of x.	<code>fruits.index('mango')</code>
<code>count(x)</code>	Counts how many times x appears.	<code>nums.count(5)</code>

<code>clear()</code>	Removes ALL items from the list.	<code>fruits.clear()</code>
<code>copy()</code>	Returns a copy of the list.	<code>b = fruits.copy()</code>
<code>extend(lst)</code>	Adds all items of another list.	<code>fruits.extend(more)</code>
<code>len(list)</code>	Returns number of items. (built-in fn)	<code>len(fruits)</code>

Live Example – Using List Methods

```
marks = [85, 72, 90, 65, 88]

marks.append(95)          # [85, 72, 90, 65, 88, 95]

marks.sort()              # [65, 72, 85, 88, 90, 95]

marks.reverse()           # [95, 90, 88, 85, 72, 65]

marks.remove(72)          # [95, 90, 88, 85, 65]

print(marks.index(90))    # 1  (position of 90)

print(len(marks))         # 5
```

6. Simple Algorithms on Lists

An **algorithm** is a step-by-step method to solve a problem. Below are four fundamental algorithms you must know:

6.1 Sum of a List

Add all the elements together. Start with total = 0 and keep adding each element.

```
# Method 1 - Using a loop (manual)

nums = [10, 20, 30, 40, 50]

total = 0

for n in nums:
    total = total + n

print('Sum:', total)      # Sum: 150

# Method 2 - Using built-in sum()

print('Sum:', sum(nums))  # Sum: 150
```

6.2 Average (Mean) of a List

Average = Sum of all elements ÷ Number of elements.

```
def calculate_average(numbers):  
    total    = sum(numbers)  
    count    = len(numbers)  
    average  = total / count  
    return average  
  
marks = [80, 90, 70, 85, 75]  
  
print('Average:', calculate_average(marks))    # Average: 80.0
```

6.3 Maximum (Largest Value)

Assume the first item is the largest. Compare with every other item and update if a bigger one is found.

```
# Method 1 - Using a loop  
  
def find_max(numbers):  
    maximum = numbers[0]    # assume first is biggest  
    for n in numbers:  
        if n > maximum:  
            maximum = n    # update if bigger found  
    return maximum  
  
print(find_max([3, 9, 1, 7, 5]))    # 9  
  
# Method 2 - Built-in  
print(max([3, 9, 1, 7, 5]))    # 9
```

6.4 Minimum (Smallest Value)

Same logic as maximum, but look for a *smaller* value.

```
# Method 1 - Using a loop  
  
def find_min(numbers):  
    minimum = numbers[0]    # assume first is smallest
```

```
    for n in numbers:

        if n < minimum:

            minimum = n

    return minimum

print(find_min([3, 9, 1, 7, 5])) # 1

# Method 2 - Built-in

print(min([3, 9, 1, 7, 5]))      # 1
```

■ **Quick Tip:** Python provides `sum()`, `max()`, `min()`, and `len()` as built-in functions so you rarely need to write these from scratch. But learning the loop version helps you understand *how* they work!

7. Complete Example – Student Marks Processor

This program puts everything together: a list, functions, and algorithms.

```
# Student marks processor

def get_summary(marks):

    total    = sum(marks)

    average = total / len(marks)

    highest  = max(marks)

    lowest   = min(marks)

    return total, average, highest, lowest

student_marks = [78, 85, 92, 60, 74, 88]

t, avg, high, low = get_summary(student_marks)

print('Total    :', t)      # 477

print('Average  :', avg)    # 79.5

print('Highest  :', high)   # 92

print('Lowest   :', low)    # 60
```

8. Applications

Application Area	How Functions + Lists Are Used
Data Processing	Store data in lists, use functions to filter, sort, and analyse.
Simple Calculations	Sum, average, max, min on lists of numbers (marks, sales, temperatures).
Modular Programming	Split a large program into small functions – easier to test and reuse.
Search & Sort	Write search or sort functions that work on any list.
Report Generation	Process a list of records and print a formatted summary.

9. Quick Revision Cheat-Sheet

Concept	Key Points
Function	def name(): ... – reusable block of code
Parameters	Variables inside () in def – placeholders for input
Arguments	Actual values passed when calling the function
return	Sends a result back; function stops after return
List	[] – ordered, mutable, allows duplicates
Indexing	list[0] = first item; list[-1] = last item
Slicing	list[start:end] – get a sub-list
append / remove	Add to end / remove first occurrence
sort / reverse	Sort ascending / flip order in-place
sum()	Returns total of all numeric items
max() / min()	Returns largest / smallest item
len()	Returns number of items in a list

■ **Quick Tip:** Practice tip: After studying each section, close your notes and try to write a small program from memory. That is the fastest way to remember Python!